

ENTWICKLERDOKUMENTATION

Technisches Systemhandbuch & Architekturspezifikation

Ausbildungsberuf: Fachinformatiker/in für Anwendungsentwicklung

Projektbezeichnung:	Smart Restaurant Adminsoftware
Auftragnehmer / Team:	Another Great Solution GmbH (Team „Der Dreier“)
Architekturmuster:	3-Schichten-Architektur (Model-View-DAO)
Technologie-Stack:	Java 17 (OpenJDK), Java Swing, SQLite 3, JDBC
Dokumentenstand & Version:	01. Oktober 2026 Version 1.2 (Final)
Status:	Produktiv / Vollständig implementiert

1. Systemarchitektur & Entwurfsprinzipien

Die Anwendung **Smart Restaurant Admin** ist als klassische Desktop-Anwendung nach dem Prinzip der **3-Schichten-Architektur** konzipiert. Ziel ist eine strikte Entkopplung zwischen Präsentationsschicht, Geschäftslogik und Datenhaltungsschicht:



Datenspeicher:	SQLite 3 Engine (smart_restaurant.db)
----------------	---------------------------------------

2. Projekt- & Paketstruktur

Das Java-Projekt ist im Paket `com.smartrestaurant` modular organisiert:

Klasse / Datei	Zweck & funktionale Verantwortung
Main.java	Einstiegspunkt; setzt System-Look-and-Feel, initialisiert Datenbank und startet WebServer & GUI.
AdminGUI.java	Swing-Hauptfenster mit 5 TabbedPane (Mitarbeiter, Artikel, Tische, Bestellungen, Systemstatus).
DatabaseManager.java	Data Access Object (DAO); kapselt alle SQL-Befehle via PreparedStatement und Transaktionen.
WebServer.java	Eingebetteter Java-HTTP-Server (Port 8080); liefert das mobile Web-Dashboard (web/index.html) aus.
Mitarbeiter.java	POJO-Domänenmodell für Personalstammdaten mit Rollenkonzept.
Artikel.java	POJO-Domänenmodell für Speisen und Getränke inklusive Preiskalkulation.
Tisch.java	POJO-Domänenmodell für Gastraumtische mit Sitzplatzkapazitäten und Status.
Bestellung.java	POJO-Domänenmodell für Bestellungen mit Fremdschlüssel-Auflösung und Gesamtsumme.
DatabaseTest.java	Automatisierter Testsuite-Runner für Continuous Integration und Regressionsprüfung.

3. Datenbankmodell & Relationales Schema (3NF)

Das Datenbankschema erfüllt strikt die Anforderungen der **3. Normalform (3NF)**. Sämtliche Relationen sind atomar, voll vom Primärschlüssel abhängig und frei von transitiven Abhängigkeiten.

SQLite DDL-Spezifikation (Auszug):

```
-- Tabelle: Mitarbeiter
CREATE TABLE IF NOT EXISTS mitarbeiter (
    mitarbeiter_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name            TEXT NOT NULL,
    benutzername    TEXT UNIQUE NOT NULL,
    rolle           TEXT NOT NULL,
    aktiv           INTEGER DEFAULT 1
);

-- Tabelle: Tisch
CREATE TABLE IF NOT EXISTS tisch (
    tisch_id        INTEGER PRIMARY KEY AUTOINCREMENT,
    tischnummer     INTEGER UNIQUE NOT NULL,
    kapazitaet      INTEGER NOT NULL,
    status          TEXT NOT NULL DEFAULT 'frei'
);

-- Tabelle: Artikel
CREATE TABLE IF NOT EXISTS artikel (
    artikel_id      INTEGER PRIMARY KEY AUTOINCREMENT,
    name            TEXT NOT NULL,
    kategorie       TEXT NOT NULL,
    preis           REAL NOT NULL,
    aktiv           INTEGER DEFAULT 1
);

-- Tabelle: Bestellung
CREATE TABLE IF NOT EXISTS bestellung (
    bestellung_id   INTEGER PRIMARY KEY AUTOINCREMENT,
    tisch_id        INTEGER NOT NULL,
    erstellt_von    INTEGER NOT NULL,
    status          TEXT NOT NULL DEFAULT 'aufgegeben',
    erstellt_am     TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    gesamtpreis     REAL DEFAULT 0.0,
    FOREIGN KEY(tisch_id) REFERENCES tisch(tisch_id),
    FOREIGN KEY(erstellt_von) REFERENCES mitarbeiter(mitarbeiter_id)
);
```

4. Sicherheitskonzept & Clean Code

Gemäß OWASP Top 10 Best Practices werden Sicherheitsrisiken durch saubere Programmiertechniken eliminiert:

- **PreparedStatement statt String-Konkatenation:** Jeder dynamische SQL-Befehl bindet Parameter über Platzhalter (?), wodurch SQL-Injections technisch unmöglich gemacht werden.
- **Ressourcensicherheit:** Konsequente Verwendung von `try-with-resources` garantiert das automatische Schließen von JDBC-Connections und Statements.
- **Thread-Sicherheit in Swing:** Alle GUI-Operationen werden über `SwingUtilities.invokeLater()` im Event Dispatch Thread ausgeführt.

5. Kompilierung & Build-Befehle

```
# 1. Quellcode mit UTF-8 kompilieren:
javac -encoding UTF-8 -cp "lib/sqlite-jdbc.jar;src" -d bin src/com/smartrestaurant/*.java

# 2. Anwendung starten:
java -cp "lib/sqlite-jdbc.jar;bin" com.smartrestaurant.Main
```

```
# 3. Automatisierte Testsuite ausführen:  
java -cp "lib/sqlite-jdbc.jar;bin" com.smartrestaurant.DatabaseTest
```

6. Wartung & Backup-Strategie

Da SQLite als dateibasierte Datenbank konzipiert ist, erfolgt die Datensicherung denkbar einfach: Die Datei `smart_restaurant.db` kann im laufenden Betrieb oder per nächtlichem Cronjob/Task-Scheduler kopiert werden. Ein vollständiger DDL/DML-Dump liegt zusätzlich als `smart_restaurant_dump.sql` vor.